PLATAFORMA DE INVERSIÓN

Guía de Diseño y Construcción

De la idea al sistema — sin código, sin jerga innecesaria

Este documento explica QUÉ construimos, POR QUÉ cada decisión y CÓMO se conectan las piezas entre sí.

El módulo de Backtesting se desarrolla al máximo detalle.
El resto de módulos se describe a nivel de diseño y propósito.

Índice

1. Para qué sirve este sistema	La visión completa de la plataforma
2. Principios de diseño	Las reglas que no se negocian jamás
3. Los cinco módulos	Una mirada general a cada sección
4. El módulo de Backtesting	Diseño completo y en detalle
4.1 Qué es un backtesting	La analogía del libro de historia
4.2 De dónde vienen los datos	Cómo entran y qué se valida
4.3 Las estrategias	Qué son y cómo se escriben
4.4 El motor de simulación	El corazón del sistema
4.5 Las salidas de un trade	Cómo y cuándo se cierra una operación
4.6 La optimización	Cómo encuentra los mejores parámetros
4.7 Los resultados	Qué se genera y cómo se interpreta
4.8 La configuración	El único archivo que el usuario edita
4.9 El orden de carpetas	Cómo se organiza todo en disco
5. El Portfolio Tracker	Seguimiento del capital real
6. El Asset Scanner	Vigilancia automática de activos
7. Análisis y Trading	El gráfico interactivo con estrategias
8. Noticias y Calendario	Contexto y notas personales
9. La tecnología elegida	Por qué cada herramienta
10. Hoja de ruta	En qué orden construimos cada cosa

1. Para qué sirve este sistema

Imagina que un fondo de inversión profesional tiene un departamento de tecnología cuyo único trabajo es encontrar estrategias rentables, vigilar el capital en todo momento y no perder nunca de vista lo que ocurre en los mercados. Este sistema es exactamente eso, pero construido para un trader individual.

La plataforma no es una herramienta para hacer una cosa. Es un ecosistema con cinco piezas que se complementan:

Módulo	Qué hace	Cuándo se usa
Backtesting	Prueba estrategias en datos históricos para ver si funcionan antes de arriesgar dinero real.	Antes de operar. Es la prioridad número 1.
Portfolio Tracker	Lleva la cuenta de dónde está el dinero: cuánto tienes, cuánto está invertido y en qué.	Todos los días, para no perder el control del capital.
Asset Scanner	Analiza activos automáticamente y les pone una nota según lo cerca que están de una zona interesante.	Para no tener que mirar cien gráficos a mano.
Análisis	Muestra un gráfico interactivo donde puedes ver estrategias ya probadas funcionando en tiempo real.	Cuando quieres tomar decisiones de trading concretas.
Noticias y Notas	Recoge noticias de los activos que sigues, eventos del calendario y permite añadir notas propias.	Para no operar a ciegas sobre el contexto.

LA GRAN IDEA Estas cinco piezas son independientes entre sí. Si el Scanner falla, el Backtesting sigue funcionando. Si cambias cómo se calcula la nota del Scanner, no tocas ni una línea del Portfolio Tracker. Esto no es un accidente de diseño: es la decisión más importante de toda la arquitectura.

2. Principios de diseño

Antes de describir cómo funciona cada módulo, hay que entender las reglas que gobiernan cada decisión. Si en algún momento una solución viola uno de estos principios, se replantea la solución, no el principio.

Todo separado, nada mezclado

Cada módulo vive en su propia carpeta, habla su propio idioma interno y no sabe nada de los demás módulos. Si necesitan compartir información, lo hacen a través de archivos (como pasar una nota escrita en lugar de hablar directamente). Esto tiene un nombre técnico —modularidad— pero la idea es simple: que cambiar una pieza no rompa las demás.

> **EJEMPLO** Si mañana decides cambiar el sistema de puntuación del Scanner, el Backtesting no se entera. No hay que tocar nada más que el archivo del Scanner.

Sin trampa en el backtest

Esta es la regla más importante del módulo de Backtesting, y vale la pena explicarla bien. Cuando probamos una estrategia en datos históricos, existe la tentación —a veces sin darse cuenta— de que el sistema "mire al futuro". Por ejemplo: calcular una media móvil usando el precio de cierre de la vela actual para decidir si entrar en esa misma vela.

Eso es trampa. En la realidad, cuando la vela cierra ya no puedes entrar a ese precio. El sistema entra siempre en el precio de apertura de la vela siguiente, nunca antes. Esto se verifica en cada parte del código y los resultados son auditables.

> **ANALOGÍA** Es como apostar en una carrera de caballos mirando la foto de llegada. Los resultados del backtest serían perfectos, pero completamente inútiles en la realidad.

Velocidad como requisito, no como mejora

El sistema está diseñado para explorar miles de combinaciones de parámetros. Si cada prueba tarda un segundo, explorar diez mil combinaciones tarda casi tres horas. Si cada prueba tarda un milisegundo, tarda diez segundos. La diferencia no es de comodidad: es de lo que es posible hacer o no.

Por eso el corazón del motor de simulación está escrito en Rust, un lenguaje de sistemas que es entre cien y mil veces más rápido que Python en operaciones como esta. Python hace el trabajo de preparar los datos y orquestar el proceso; Rust hace el trabajo pesado de simular trade a trade.

Un solo sitio para cada configuración

Cambiar las fechas del backtest, el activo que se prueba, las comisiones o el número de pruebas: todo se hace en un único archivo llamado config.py. Nada está enterrado en el código de la lógica. Si algo necesita cambiar, el usuario sabe exactamente dónde ir.

Mismos datos, mismo resultado

Si ejecutas el sistema dos veces con los mismos parámetros y el mismo número de semilla aleatoria, obtienes exactamente los mismos resultados. Esto permite comparar variaciones de forma fiable: si cambias un parámetro y el resultado mejora, sabes que es por ese cambio y no por azar.

Construido para crecer sin romperse

El sistema empieza en local y en terminal. Pero cada decisión de diseño contempla que en el futuro pueda subirse a un servidor, tener una interfaz web y conectar con fuentes de datos externas. Cuando llegue ese momento, no habrá que reescribir la lógica central: solo añadir capas encima.

3. Los cinco módulos: mirada general

Esta sección da una visión de helicóptero de cada módulo. Los detalles vienen después, especialmente para el Backtesting que es la prioridad número uno.

> **ORDEN DE CONSTRUCCIÓN** Los módulos se construyen en orden de prioridad. El Backtesting va primero porque sin una estrategia probada, los demás módulos no sirven de nada. El Portfolio Tracker, el Scanner, el Análisis y las Noticias vienen después, una vez el backtesting funciona sólidamente.

Módulo 1: Backtesting

Es el laboratorio donde se prueban las estrategias. Le dices al sistema qué activo, qué período de tiempo, qué estrategia y qué parámetros quieres probar, y él simula qué habría pasado si hubieras operado con esa estrategia durante ese período. Lo hace miles de veces variando los parámetros para encontrar cuáles funcionan mejor.

Módulo 2: Portfolio Tracker

Es el cuaderno de contabilidad del sistema. Responde preguntas como: ¿cuánto capital tengo en total? ¿Cuánto está en inversiones de largo plazo y en cuáles? ¿Cuánto está en trading a corto plazo? ¿Cuánto me queda disponible? ¿Cómo ha evolucionado mi capital este mes? Todo esto sin que tengas que calcularlo tú a mano.

Módulo 3: Asset Scanner

Es el vigía automático. Le dices qué activos quieres seguir y él los analiza continuamente, asignándoles una nota del 0 al 100 según lo cerca que están de una zona interesante para operar. Cuando un activo llega a una nota alta, te lo indica. Así no tienes que mirar gráficos manualmente todo el día.

La nota varía según la temporalidad: el mismo activo puede tener una nota alta en el gráfico de 4 horas y baja en el diario. Ambas son válidas y el sistema te permite ver las dos.

Módulo 4: Análisis y Trading

Es la mesa de trabajo donde se toman las decisiones finales. Muestra un gráfico interactivo de cualquier activo con el timeframe que elijas. Puedes añadir indicadores sueltos para explorar, o cargar directamente una de las estrategias que ya has probado en backtesting y ver cómo se comporta en el mercado actual.

Módulo 5: Noticias y Calendario

Es el contexto. Recoge las noticias relevantes de los activos que estás siguiendo, muestra un calendario con eventos importantes (presentaciones de resultados empresariales, decisiones de tipos de interés, datos macro) y te permite añadir tus propias notas: "entré en AAPL el 15 de marzo, revisar en 90 días" o "BTC presenta soporte clave en 60.000, observar".

4. El Módulo de Backtesting

Diseño completo — la prioridad número uno

4.1 Qué es un backtesting y por qué importa tanto

Imagina que tienes una estrategia para invertir en bolsa: "compro cuando el RSI cae por debajo de 30 y vendo cuando sube por encima de 70". Antes de arriesgar dinero real con esta idea, querrías saber: ¿habría funcionado esta estrategia en los últimos tres años?

Un backtesting te responde esa pregunta. Coge los datos históricos del activo, recorre cada vela del pasado y simula exactamente qué habría pasado si hubieras operado con esa estrategia: cuándo habrías entrado, cuándo habrías salido, cuánto habrías ganado o perdido, cuál habría sido el peor momento (drawdown máximo) y muchas métricas más.

> **ANALOGÍA** Es como un piloto de aviación que entrena en un simulador antes de volar de verdad. Los resultados del simulador no garantizan el éxito en el vuelo real, pero sí te dicen si la estrategia tiene algún sentido o si habría arruinado la cuenta en cuestión de semanas.

El problema es que probar una estrategia con unos parámetros concretos no es suficiente. Quizás con RSI de 14 períodos los resultados son mediocres, pero con RSI de 9 son excelentes. O quizás el stop loss óptimo no es del 20% del capital sino del 15%. Para encontrar la combinación óptima hay que probar miles de variaciones, y eso es exactamente para lo que está diseñado este módulo.

4.2 De dónde vienen los datos y cómo se preparan

La carpeta de datos

El usuario es responsable de sus propios datos. Los deposita en una carpeta llamada "datos" dentro del proyecto. El sistema no descarga datos por sí mismo en esta versión, aunque el diseño contempla añadir esa función en el futuro.

Los archivos siguen una convención de nombres sencilla: el nombre del activo seguido del timeframe mínimo en que están guardados. Por ejemplo, los datos de Bitcoin en velas de 1 minuto se llaman BTC_1m. Los datos de oro en 1 minuto, GOLD_1m. Así el sistema siempre sabe qué archivo cargar para cada activo.

> **FORMATO RECOMENDADO** El formato preferido es Parquet, un tipo de archivo diseñado para datos en columnas que se lee mucho más rápido que un CSV. No obstante, el sistema acepta también CSV y Feather por si el usuario tiene los datos en esos formatos.

Lo mínimo que deben tener los datos

Para que una estrategia pueda funcionar, cada vela del archivo debe tener como mínimo cinco valores: la hora exacta de la vela (timestamp), el precio de apertura (open), el precio más alto (high), el precio más bajo (low) y el precio de cierre (close). El volumen es opcional pero muy recomendable.

Algunas estrategias más avanzadas pueden necesitar datos adicionales, como el volumen de compra agresiva (taker buy volume) o el interés abierto. El sistema está diseñado para aceptar cualquier columna adicional, siempre que la estrategia que la usa lo declare explícitamente. Si los datos no tienen esa columna, el sistema avisa con claridad antes de empezar, en lugar de fallar misteriosamente a mitad de la ejecución.

Qué valida el sistema al cargar datos

Antes de empezar cualquier backtest, el sistema realiza una serie de comprobaciones automáticas para asegurarse de que los datos son válidos:

- Que el archivo existe para el activo y timeframe que se ha pedido.
- Que tiene todas las columnas mínimas (timestamp, open, high, low, close).
- Que no hay huecos ni valores vacíos en los precios.
- Que las velas están en orden cronológico de más antigua a más reciente.
- Que no hay velas duplicadas con el mismo timestamp.
- Que la estrategia elegida tiene todas las columnas adicionales que necesita.

> **SI ALGO FALLA** El sistema para inmediatamente y muestra un mensaje claro que dice exactamente qué falta y por qué. No sigue adelante con datos malos y resultados potencialmente falsos.

El resampleo de timeframes

Los datos siempre se almacenan en el timeframe más pequeño disponible: normalmente velas de 1 minuto. Esto tiene sentido porque a partir de velas de 1 minuto puedes construir velas de 5 minutos, 15 minutos, 1 hora o cualquier timeframe mayor. Pero no al revés: si solo tienes velas de 15 minutos, no puedes reconstruir las de 5 minutos.

Cuando la estrategia opera en un timeframe mayor, el sistema agrupa las velas automáticamente. Este proceso se llama resampleo y sigue reglas muy precisas: el precio de apertura de la vela grande es el de la primera vela pequeña, el máximo es el mayor de todas las velas pequeñas, el mínimo es el menor, el cierre es el de la última y el volumen es la suma de todas.

REGLA FUNDAMENTAL Solo se puede ir hacia timeframes más grandes, nunca más pequeños. Si el usuario pide datos de 5 minutos teniendo solo datos de 15 minutos, el sistema lo detecta y explica el problema claramente en lugar de inventar datos inexistentes.

El filtro de fechas

Es habitual tener un archivo con varios años de datos pero querer hacer el backtest solo sobre un período concreto. El sistema permite definir una fecha de inicio y una fecha de fin en la configuración. Solo ese período se carga en memoria para trabajar, aunque el archivo tenga muchos más datos. Esto ahorra memoria y permite hacer comparaciones entre períodos: ¿funcionó mejor la estrategia en 2022 que en 2023?

4.3 Las estrategias

Qué es una estrategia en este sistema

Una estrategia es un archivo de código que contiene dos cosas: la lógica que decide cuándo entrar en una operación (señales de entrada) y, opcionalmente, la lógica que decide cuándo salir (señales de salida personalizadas). Cada estrategia vive en su propio archivo independiente dentro de la carpeta de estrategias.

El sistema reconoce automáticamente todas las estrategias que hay en esa carpeta. No hay que registrarlas manualmente ni tocar ningún archivo central. Simplemente creas el archivo y el sistema lo incluye al iniciarse.

El contrato de una estrategia

Toda estrategia debe cumplir un contrato mínimo: tener un identificador numérico único, un nombre, y dos funciones. La primera función define el espacio de búsqueda para la optimización (qué parámetros tiene la estrategia y en qué rango se pueden mover). La segunda función recibe los datos del activo y devuelve, para cada vela, si hay señal de entrada long, señal de entrada short, o ninguna.

EJEMPLO SENCILLO Una estrategia de cruce de RSI diría: 'mis parámetros son el período del RSI (entre 7 y 28) y el umbral de sobrecompra (entre 65 y 80). Para cada vela, hay señal long si el RSI estaba por debajo del umbral de sobreventa y acaba de cruzarlo hacia arriba. Hay señal short si estaba por encima del umbral de sobrecompra y lo cruza hacia abajo'.

La regla de oro: nunca mirar al futuro

Es la regla más importante de cualquier estrategia y vale la pena repetirla. Al calcular un indicador para la vela actual, solo puedes usar datos de velas anteriores. El precio de cierre de la vela actual no se conoce hasta que la vela cierra, así que no puedes usarlo para decidir si entras en esa misma vela.

Lo que sí puedes hacer es: cuando la vela cierra y genera una señal, entrar en el precio de apertura de la vela siguiente. Eso sí es posible en la realidad. El sistema garantiza esto en su motor de simulación: la entrada siempre ocurre en el precio de apertura de la vela posterior a la señal, nunca antes.

> **POR QUÉ IMPORTA TANTO** Un backtest que mira al futuro puede mostrar resultados espectaculares completamente falsos. El sistema está diseñado para que esto sea imposible por construcción, no por disciplina del usuario.

Indicadores disponibles

La clase base de la que heredan todas las estrategias incluye los indicadores más comunes ya implementados de forma eficiente: medias móviles simples y exponenciales, RSI, ATR (rango verdadero promedio) y Bandas de Bollinger, entre otros. El usuario puede usar estos directamente o añadir sus propios cálculos. Todo está vectorizado, lo que significa que se calcula para todas las velas al mismo tiempo en lugar de una por una, lo cual es mucho más rápido.

Estrategias con salidas personalizadas

Por defecto, el sistema gestiona cuándo salir de un trade usando las reglas configuradas globalmente (stop loss, take profit o número de velas). Pero una estrategia puede tener su propia lógica de salida: por ejemplo, salir cuando el RSI cruza la zona media desde la dirección de la posición.

En ese caso, la estrategia incluye en su salida de datos dos columnas adicionales: una que indica en qué velas hay señal de salida long y otra para señal de salida short. El motor de simulación las respeta y cierra el trade cuando detecta la señal. El stop loss sigue activo como red de seguridad de último recurso.

4.4 El motor de simulación

Qué hace el motor

El motor de simulación es el corazón del sistema. Su trabajo es recorrer todas las velas del período de backtest, detectar las señales de entrada de la estrategia, abrir trades, seguirlos vela a vela hasta que se activa la condición de salida, calcular el resultado de cada trade y actualizar el saldo de la cuenta.

Este proceso parece simple pero se repite potencialmente decenas de millones de veces en un solo run de optimización con muchos trials y muchas velas. Por eso está escrito en Rust, un lenguaje de programación de sistemas que genera código tan eficiente como el de los sistemas

operativos. Python prepara todos los datos y le pasa el trabajo pesado a Rust, que lo ejecuta en microsegundos.

El flujo exacto de un trade

Entender cómo se ejecuta un trade en el motor es fundamental para confiar en los resultados:

1. La estrategia detecta una señal de entrada en la vela N (por ejemplo, RSI cruza hacia arriba).
2. El motor anota que hay señal, pero no entra todavía.
3. En la vela N+1, el motor entra al precio de apertura. Este es el único momento en que una entrada es realista: la vela anterior ya cerró y la nueva acaba de abrir.
4. Desde ese momento, el motor vigila la posición vela a vela hacia adelante.
5. En cada vela comprueba si se ha tocado el stop loss (mirando el mínimo de la vela para longs, el máximo para shorts), si se ha tocado el take profit, si la estrategia ha generado señal de salida, o si se ha alcanzado el número máximo de velas configurado.
6. Cuando se activa cualquiera de estas condiciones, el motor cierra el trade al precio correspondiente.
7. Calcula el beneficio o pérdida, descuenta las comisiones y actualiza el saldo de la cuenta.
8. Pasa a la siguiente señal. Si hay señales mientras el trade está abierto, las ignora.

> **DETALLE IMPORTANTE** Si el stop loss se toca dentro de una vela, el precio de salida es el precio del stop loss, no el de cierre de la vela. Esto simula de forma más realista lo que ocurriría en la realidad, donde el stop se ejecuta en cuanto el precio lo alcanza.

La gestión del capital

El motor no opera con el capital total de la cuenta. Opera con un colateral por trade que el usuario define en la configuración. Sobre ese colateral se aplica un apalancamiento para determinar el tamaño real de la posición.

Por ejemplo, si el colateral por trade es 500 dólares y el apalancamiento es 10x, la posición real que se abre es de 5.000 dólares. Si el precio sube un 1%, el beneficio es de 50 dólares (un 10% sobre el colateral). Si el stop loss está al 20% del colateral, se activará cuando la posición haya perdido 100 dólares.

El sistema también tiene un saldo mínimo operativo. Si el saldo de la cuenta cae por debajo de esa cifra, el backtest se detiene automáticamente. Esto simula lo que ocurriría en la realidad si una racha de pérdidas dejara la cuenta sin margen suficiente para seguir operando.

SOBRE LAS COMISIONES Las comisiones se calculan como un porcentaje sobre el volumen total operado. Por defecto se aplican tanto en la apertura como en el cierre de cada trade. Ambas opciones son configurables.

4.5 Las salidas de un trade

Por qué las salidas son tan importantes

Muchos traders se centran demasiado en las entradas y poco en las salidas. Pero la realidad es que saber cuándo salir de un trade es igual o más importante que saber cuándo entrar. Una buena entrada con una mala salida puede convertirse en pérdida. Una entrada mediocre con una buena salida puede ser rentable.

Por eso el sistema de salidas está diseñado de forma modular: cada tipo de salida es independiente y se puede combinar con otros. Añadir un nuevo tipo de salida en el futuro (trailing stop, salida por volatilidad, etc.) no requiere tocar el motor ni las estrategias existentes.

Tipo 1: Stop Loss y Take Profit fijos

El tipo más simple y el que se implementa primero. Se define un porcentaje máximo de pérdida sobre el colateral (stop loss) y un porcentaje de ganancia objetivo (take profit). Cuando cualquiera de los dos se alcanza, el trade se cierra automáticamente.

Es importante entender que estos porcentajes son sobre el colateral, no sobre el precio. Si el colateral es 500 dólares y el stop loss está al 20%, el trade se cierra cuando la pérdida llega a 100 dólares. Esto facilita el control del riesgo porque el usuario piensa en términos de dinero, no de movimientos de precio que dependen del apalancamiento.

POSIBILIDAD DE OPTIMIZACIÓN El sistema puede configurarse para que Optuna también busque los valores óptimos de stop loss y take profit, en lugar de usar valores fijos. Esto se activa con un parámetro en la configuración.

Tipo 2: Salida por número de velas

Una alternativa al stop loss clásico. En lugar de salir cuando el precio llega a cierto nivel, el trade se cierra automáticamente después de un número fijo de velas. Útil para estrategias con un horizonte temporal definido: "opero durante máximo 12 velas de 1 hora y luego cierro pase lo que pase".

Este tipo de salida también se puede combinar con un stop loss: si el stop loss se activa antes de llegar al número de velas máximo, cierra el trade de todas formas. El primero que ocurra tiene preferencia.

Tipo 3: Salida personalizada de la estrategia

Algunas estrategias tienen su propia lógica para decidir cuándo salir, basada en los mismos indicadores que usan para entrar. En este caso, la estrategia incluye en su código las condiciones de salida, y el motor las respeta.

Cuando una estrategia define sus propias salidas, el take profit estándar se desactiva automáticamente (porque la estrategia decide cuándo tomar beneficios). El stop loss permanece activo como protección de emergencia ante movimientos extremos que la lógica de salida no hubiera podido gestionar.

El modo ALL

Existe una opción especial en la configuración que ejecuta el backtest con cada tipo de salida por separado y guarda los resultados de forma independiente. Esto permite comparar directamente: ¿funciona mejor esta estrategia con stop loss fijo o con salida por velas?

4.6 La optimización

El problema que resuelve

Una estrategia de RSI tiene al menos dos parámetros: el período del RSI y el umbral de sobrecompra/sobreventa. Si el período puede ir de 7 a 28 (22 posibles valores) y el umbral de 60 a 80 en pasos de 1 (21 posibles valores), hay 462 combinaciones posibles solo para esos dos parámetros. Con cinco o diez parámetros el espacio crece exponencialmente.

Probar todas las combinaciones una por una sería inviable. La optimización con Optuna resuelve esto de forma inteligente: en lugar de explorar el espacio al azar o de forma exhaustiva, aprende de los resultados anteriores para guiar la búsqueda hacia las zonas más prometedoras.

Los tres modos de búsqueda

El sistema ofrece tres modos de optimización seleccionables desde la configuración:

QMC (Quasi-Monte Carlo)	Usa secuencias matemáticas especiales llamadas secuencias de Sobol para distribuir las pruebas de forma equidistante por todo el espacio de parámetros. No aprende de los resultados anteriores: simplemente se asegura de cubrir el espacio de forma uniforme, sin dejar zonas sin explorar ni acumular pruebas en zonas que ya se han visto. Ideal para una exploración inicial exhaustiva.
TPE (Tree-structured Parzen Estimator)	El modo inteligente. Aprende de cada trial anterior para estimar qué zonas del espacio de parámetros tienen más probabilidad de dar buenos resultados. Dedica más intentos a explorar zonas prometedoras. Ideal para refinar después de una exploración inicial.

HYBRID	La combinación óptima para la mayoría de casos. La primera mitad de los trials usa QMC para mapear el terreno uniformemente; la segunda mitad usa TPE para explotar las mejores zonas encontradas. Es el modo recomendado por defecto.

Cómo sabe el sistema qué trial es mejor

Después de cada trial, el sistema calcula una puntuación que indica qué tan bueno fue ese conjunto de parámetros. En esta primera versión, la puntuación es el ROI (retorno sobre la inversión): cuánto porcentaje ganó la estrategia en ese período con esos parámetros.

Esta función de puntuación está completamente separada del resto del sistema. Cuando quieras reemplazarla por algo más sofisticado (que tenga en cuenta el drawdown, la consistencia, la frecuencia de operaciones, etc.), solo hay que cambiar ese archivo. Nada más del sistema necesita saber que cambió.

> **SOBRE LA REPRODUCIBILIDAD** Si defines un número de semilla aleatoria en la configuración, dos ejecuciones con los mismos parámetros producen exactamente los mismos resultados. Si dejas la semilla en modo aleatorio, los resultados varían ligeramente entre ejecuciones, lo cual puede ser útil para detectar si una estrategia es robusta o solo funcionó bien en una combinación específica.

Cuántos trials hacer

El número de trials que vale la pena ejecutar depende de cuántos parámetros tiene la estrategia. Como regla general, con dos o tres parámetros son suficientes 64 o 128 trials. Con cinco a ocho parámetros conviene ir a 256 o 512. Para el modo QMC, se recomienda usar potencias de 2 (64, 128, 256, 512...) porque las propiedades matemáticas de la secuencia de Sobol se aprovechan mejor así.

El paralelismo: velocidad sin agotar el servidor

Los trials se pueden ejecutar en paralelo, usando varios núcleos del procesador a la vez. Cada worker procesa un trial independiente simultáneamente, multiplicando la velocidad proporcional al número de cores que se usen.

Sin embargo, los distintos activos y timeframes se procesan de forma secuencial: primero se acaba todo lo relacionado con BTC en 1 hora, se libera la memoria, y luego se empieza con BTC en 4 horas o con GOLD. Esto es más lento que procesar todo en paralelo, pero mantiene el uso de memoria bajo control independientemente de cuántos activos y timeframes se estén probando.

> **CONTROL DESDE CONFIG** El número de workers paralelos se controla con un parámetro en la configuración. La opción recomendada es usar todos los cores menos uno, para dejar el servidor responsive mientras corre la optimización.

4.7 Los resultados

Lo que se genera al terminar un run

Al acabar la optimización de una combinación de activo, timeframe y estrategia, el sistema genera dos tipos de archivos: un Excel con todos los datos y estadísticas, y archivos HTML interactivos con gráficos para los mejores resultados. Ambos son opcionales y se pueden desactivar desde la configuración.

El Excel de resultados

El archivo Excel tiene tres pestañas:

Pestaña RESUMEN	Una fila por cada trial completado, ordenadas de mejor a peor resultado. Incluye el score, el ROI, el win rate, el profit factor, el sharpe ratio, el drawdown máximo, el número de trades, la duración media de las posiciones y los valores exactos de todos los parámetros que Optuna eligió para ese trial. Permite comparar todos los trials de un vistazo y filtrar por cualquier métrica.
Pestaña TRADES	Todos los trades ejecutados por el mejor trial. Para cada operación muestra: cuándo entró y cuándo salió, a qué precio, si fue long o short, cuánto ganó o perdió, qué comisiones pagó y cuál fue el saldo de la cuenta después. Permite auditar cualquier operación y verificar que el backtest se comportó como era de esperar.
Pestaña EQUITY	La curva de capital del mejor trial. Una fila por cada trade cerrado con el saldo resultante. Facilita ver cómo evolucionó el dinero a lo largo del tiempo y detectar si hubo rachas de pérdidas largas o períodos de estancamiento.

Los gráficos HTML

Para los mejores N trials (configurable), el sistema genera un archivo HTML standalone que se puede abrir en cualquier navegador. Muestra el gráfico de velas del activo con los indicadores de la estrategia superpuestos, las entradas marcadas con flechas de colores (azul para longs, naranja para shorts), las salidas marcadas, y un encabezado con las métricas más relevantes de ese trial.

Al pasar el ratón sobre una entrada o salida, aparece un tooltip con todos los detalles de esa operación: precio, resultado, duración, motivo de salida. El archivo HTML es completamente independiente, no necesita conexión a internet ni servidores para funcionar.

La organización de carpetas de resultados

Cada combinación de activo, timeframe, estrategia y tipo de salida tiene su propia carpeta de resultados. No hay mezcla de datos. Si corres la misma estrategia sobre BTC en 1 hora y sobre GOLD en 1 hora, sus resultados viven en carpetas separadas con nombres claros que indican exactamente qué contienen.

> **EJEMPLO DE ESTRUCTURA** Los resultados de la estrategia RSI_REVERSION sobre BTC en timeframe de 1 hora con salida FIXED estarían en: resultados/BTC/1h/RSI_REVERSION/FIXED/ — junto a su Excel y sus HTMLs.

4.8 La configuración

Toda la configuración de un run vive en un único archivo llamado config.py. El usuario edita este archivo para cambiar cualquier parámetro y luego ejecuta el sistema. Nada más. No hay que tocar archivos de lógica ni entender el código interno para cambiar el activo, las fechas, el número de trials o los parámetros de capital.

A continuación se describen todos los parámetros agrupados por categoría:

Activos y datos

ACTIVO	El activo o lista de activos a probar. Ejemplos: 'BTC', o ['BTC', 'GOLD', 'ETH'] para varios a la vez.
CARPETA_DATOS	La carpeta donde están los archivos de precios. Por defecto: 'datos'.
FORMATO_DATOS	El formato de los archivos: 'parquet' (recomendado), 'csv' o 'feather'.

Timeframes

TIMEFRAME	El timeframe en que opera la estrategia. Opciones: '1m', '5m', '15m', '30m', '1h', '4h', '1d'. Se puede pasar una lista para probar varios. El sistema hace el resampleo automáticamente desde los datos de 1 minuto.

Fechas

FECHA_INICIO	Fecha de inicio del período de backtest. Formato: 'AAAA-MM-DD'.
FECHA_FIN	Fecha de fin. El dataset puede tener más datos; solo se usa el rango definido aquí.

Estrategias

ESTRATEGIA_ID	El identificador numérico de la estrategia a ejecutar. Se puede pasar una lista de IDs o la palabra 'all' para ejecutar todas las estrategias registradas.

Capital y comisiones

SALDO_INICIAL	Capital inicial de la cuenta en dólares.
SALDO_USADO_POR_TRADE	Colateral destinado a cada operación individual en dólares.
APALANCAMIENTO	Multiplicador sobre el colateral para determinar el tamaño real de la posición.
SALDO_MINIMO_OPERATIVO	Si el saldo cae por debajo de este valor, el backtest se detiene.
COMISION_PCT	Comisión por operación como fracción decimal. Por ejemplo: 0.0005 equivale al 0.05%.
COMISION_LADOS	Número de lados en que se aplica la comisión. 1 = solo apertura. 2 = apertura y cierre.

Salidas

EXIT_TYPE	Tipo de salida: 'FIXED' (stop loss y take profit), 'BARS' (número de velas), 'CUSTOM' (lógica de la estrategia) o 'ALL' (todos por separado).
EXIT_SL_PCT	Stop loss como porcentaje del colateral. Ejemplo: 20 significa que se cierra cuando la pérdida llega al 20% del colateral.
EXIT_TP_PCT	Take profit como porcentaje del colateral.
EXIT_VELAS	Número máximo de velas que puede durar un trade. Solo aplica si EXIT_TYPE es 'BARS'.
OPTIMIZAR_SALIDAS	Si es verdadero, Optuna también optimiza los valores de stop loss y take profit en lugar de usar los fijos definidos aquí.

Optimización

N_TRIALS	Número de combinaciones de parámetros que Optuna va a probar. Potencias de 2 (64, 128, 256, 512...) para el modo QMC.
OPTUNA_SAMPLER	Modo de búsqueda: 'QMC' para exploración uniforme, 'TPE' para búsqueda guiada por resultados, 'HYBRID' para combinar ambos (recomendado).
OPTUNA_SEED	Número de semilla para reproducibilidad. Dejar en blanco (None) para resultados diferentes en cada ejecución.

Paralelismo

N_JOBS	Número de trials que corren en paralelo. -1 usa todos los cores, -2 usa todos menos uno (recomendado), 1 es modo secuencial (útil para depurar problemas).

Resultados y reporting

USAR_EXCEL	Si es verdadero, genera el Excel de resultados al terminar.
MAX_PLOTS	Número de gráficos HTML a generar (los mejores N por score). 0 desactiva los gráficos.
MAX_ARCHIVOS	Número máximo de archivos a conservar por carpeta de resultados. El sistema elimina los más antiguos automáticamente.
GRAFICA_RANGO	'all' muestra todo el período del trial, '2m' muestra los últimos dos meses, 'custom' permite definir un rango de fechas manual.

4.9 El orden de carpetas del módulo

Esta es la estructura completa de carpetas del módulo de backtesting. Cada carpeta tiene una responsabilidad clara y no hay código mezclado entre responsabilidades distintas:

backtesting/core/	La base: la clase madre de estrategias, los tipos de datos y el registro automático de estrategias.
backtesting/datos/	Todo lo relacionado con los datos: carga, validación y resampleo. Nada más.
backtesting/engine/	El motor de simulación escrito en Rust. Se compila una vez y se usa como si fuera una librería normal de Python.
backtesting/salidas/	Un archivo por tipo de salida: fixed.py, bars.py y custom.py. Añadir un nuevo tipo significa añadir un nuevo archivo aquí.
backtesting/optimizacion/	El runner de Optuna, la creación de estudios por sampler y la función de puntuación.
backtesting/reporting/	Un reporter por tipo de output: Rich para la terminal, Excel y HTML. Cada uno es independiente.
backtesting/estrategias/	Las estrategias del usuario. Un archivo por estrategia. El sistema las reconoce automáticamente.
backtesting/resultados/	Los outputs de cada run: Excel y HTMLs organizados por activo, timeframe, estrategia y tipo de salida.

5. El Portfolio Tracker

Seguimiento del capital real en todo momento

Para qué sirve

Una vez tienes estrategias probadas y empiezas a operar con dinero real, necesitas saber en todo momento cuánto tienes y dónde está. El Portfolio Tracker es el cuaderno de contabilidad inteligente de la plataforma.

No ejecuta operaciones por ti ni toma decisiones. Es un sistema de registro y seguimiento: tú introduces las operaciones que haces y él te da el contexto completo de tu situación financiera.

Cómo se organiza el capital

El sistema divide el capital total en tres bolsas independientes. Esta separación es importante porque el dinero que está en una inversión de seis meses no es el mismo que el que usas para trading diario, y ambos son distintos del que tienes disponible para nuevas oportunidades.

Largo plazo	Capital en posiciones de semanas o meses. Acciones, ETFs, crypto en hold. Con información de cuándo entraste, a qué precio, cuánto tienes y cuánto vale ahora.
Trading	Capital asignado a operaciones a corto plazo. Con historial de todas las operaciones, resultado acumulado y porcentaje de acierto.
Capital libre	Lo que no está invertido en ninguna de las dos bolsas anteriores. Se calcula automáticamente: total menos largo plazo menos trading.

Qué puedes ver y hacer

- Ver el capital total y su desglose en las tres bolsas de un vistazo.
- Ver todas tus posiciones activas de largo plazo: activo, fecha de entrada, precio de entrada, precio actual, cantidad y ROI.
- Ver el historial completo de operaciones de trading con todos sus detalles.
- Ver cómo ha evolucionado tu capital total a lo largo del tiempo en una gráfica.
- Registrar cuando añades dinero desde fuera del sistema o cuando mueves capital de una bolsa a otra.
- Añadir notas a posiciones específicas: recordatorios de fechas de resultados, niveles a vigilar, planes de salida.
- Ver estadísticas de tu trading: ROI acumulado, porcentaje de operaciones ganadoras, mejor y peor trade.

Diseño técnico base

El Portfolio Tracker usa una base de datos SQLite local, que es básicamente un archivo en tu ordenador que funciona como una base de datos completa. Tiene tres tablas principales:

Posiciones	Cada inversión activa o cerrada: activo, tipo (largo plazo o trading), fechas, precios, cantidad y notas.
Operaciones	Cada movimiento sobre una posición: apertura, cierre, ampliación. Con precio, cantidad, comisión y resultado.
Movimientos de capital	Depósitos, retiradas y transferencias entre bolsas. Con fecha, cantidad y concepto.

6. El Asset Scanner

Vigilancia automática — la nota de cada activo en tiempo real

Para qué sirve

Seguir manualmente veinte activos distintos mirando sus gráficos varias veces al día es inviable. El Asset Scanner automatiza esa vigilancia: analiza todos los activos que le digas y les asigna una nota del 0 al 100 que indica qué tan cerca están de una zona interesante para operar.

No te dice qué hacer. Te dice qué vale la pena mirar con más atención, para que tú tomes la decisión final.

Cómo funciona la nota

La nota de cada activo se calcula con una función que recibe los datos de precio y devuelve un número entre 0 y 100. El algoritmo que calcula esa nota está en su propio archivo y se puede cambiar sin tocar nada más del sistema.

La nota varía según el timeframe que se analice. El mismo activo en el mismo momento puede tener nota 75 en el gráfico de 4 horas (zona interesante a corto plazo) y nota 35 en el diario (sin setup claro a largo plazo). El sistema permite ver ambas y filtrar por temporalidad.

Además, la nota tiene historial: puedes ver cómo ha evolucionado en los últimos días para saber si lleva tiempo subiendo hacia zona de entrada o si acaba de salir de ella.

Qué puedes ver y hacer

- Añadir y eliminar activos de la lista de seguimiento fácilmente.
- Ver la nota actual de cada activo actualizada automáticamente cada ciertos minutos.
- Ver la curva histórica de la nota para saber si va subiendo o bajando.
- Filtrar la vista por temporalidad (1h, 4h, 1d) y ver cómo cambia la nota.
- Ver si el activo ya está en tu cartera (cruzado automáticamente con el Portfolio Tracker).
- Recibir una alerta cuando un activo supera un umbral de nota que tú defines.

7. Análisis y Trading

El gráfico interactivo donde se toman las decisiones

Para qué sirve

El Scanner te dice qué activos merecen atención. El módulo de Análisis es donde los miras en detalle para decidir si realmente hay un setup y cuándo actuar.

Es un gráfico de velas interactivo con el que puedes explorar cualquier activo, cualquier timeframe, añadir indicadores y, sobre todo, cargar directamente las estrategias que ya has validado en backtesting para ver cómo se comportan en el mercado actual.

Qué puedes ver y hacer

- Seleccionar cualquier activo disponible en tu carpeta de datos y visualizarlo en gráfico de velas.
- Cambiar el timeframe del gráfico dinámicamente.
- Añadir indicadores sueltos (medias móviles, RSI, MACD, etc.) sin necesidad de que formen parte de ninguna estrategia.
- Cargar una estrategia ya probada en backtesting y ver sus señales en tiempo real sobre el gráfico.
- Ver qué entradas y salidas habría generado la estrategia en el período visible del gráfico.

Relación con el backtesting

Este módulo no duplica código con el backtesting. Solo importa la clase de la estrategia (el archivo que contiene su lógica), no el motor de simulación. Usa los mejores parámetros encontrados en el backtesting (guardados en el Excel de resultados) para configurar la estrategia automáticamente.

> **ACLARACIÓN IMPORTANTE** Este módulo no ejecuta operaciones reales en ningún broker. Es una herramienta de análisis y visualización. La ejecución real de las órdenes es responsabilidad del trader.

8. Noticias y Calendario

El contexto que rodea las decisiones de inversión

Para qué sirve

Ninguna decisión de inversión ocurre en el vacío. Una estrategia puede generar una señal perfecta desde el punto de vista técnico justo el día antes de que la empresa presente resultados o el banco central anuncie una decisión sobre tipos de interés. Ignorar ese contexto puede ser muy caro.

Este módulo reúne en un solo lugar las noticias relevantes de los activos que sigues, el calendario de eventos importantes y tus propias notas de seguimiento.

Qué puedes ver y hacer

- Ver noticias recientes de cada activo de tu lista de seguimiento o de tu cartera.
- Ver un calendario con eventos próximos: presentaciones de resultados empresariales, decisiones de tipos de interés, publicaciones de datos macro.
- Filtrar el calendario por activo: "muéstrame todos los eventos de AAPL en los próximos 30 días".
- Añadir tus propias notas asociadas a un activo y una fecha: recordatorios, planes de seguimiento, observaciones personales.
- Ver las notas de una posición directamente desde el Portfolio Tracker, sin saltar entre secciones.

Diseño técnico base

Las noticias llegan de APIs públicas de noticias financieras. El calendario de eventos puede gestionarse manualmente al principio (el usuario lo introduce) con la posibilidad de conectar fuentes automáticas en el futuro. Todo se guarda en una base de datos SQLite local con tres tablas simples: noticias, eventos y notas personales.

9. La tecnología elegida

Las herramientas no se eligieron por moda ni por familiaridad, sino porque cada una es la mejor opción para el problema específico que resuelve. Aquí se explica el razonamiento detrás de cada elección clave.

Rust para el motor de simulación

El loop de simulación trade a trade es la operación más repetida de todo el sistema. Con 100 trials y 100.000 velas por trial, el motor ejecuta decenas de millones de operaciones simples de comparación de precios. En Python, esto puede tardar minutos. En Rust, tarda milisegundos.

Rust es un lenguaje de programación de sistemas que genera código tan eficiente como C o C++ pero sin sus problemas históricos de seguridad de memoria. La integración con Python se hace mediante PyO3, que permite que el código Rust se use exactamente como si fuera una librería Python normal.

Polars para la transformación de datos

Polars es una librería de manipulación de datos diseñada desde cero para ser rápida y eficiente en memoria. A diferencia de Pandas (la alternativa más conocida), Polars procesa los datos en columnas, aprovecha todos los núcleos del procesador automáticamente y usa una arquitectura de evaluación diferida: solo ejecuta los cálculos cuando realmente los necesitas.

Para el resampleo de OHLCV, la validación de datos y la preparación de señales, Polars es entre 5 y 20 veces más rápido que Pandas con datasets grandes. En un sistema orientado a la velocidad, esa diferencia importa.

Optuna para la optimización

Optuna es el estándar de facto para optimización de hiperparámetros en Python. Ofrece múltiples algoritmos de búsqueda (incluidos QMC y TPE), es reproducible con semilla, gestiona los estudios en bases de datos SQLite y tiene un dashboard visual para explorar resultados. Está ampliamente probado en entornos de producción.

Parquet para los datos

Parquet es un formato de archivo columnar diseñado para datos como series temporales financieras. Comprime muy bien los datos numéricos y se puede leer parcialmente (solo las columnas que necesitas) sin cargar el archivo completo. Comparado con CSV, es entre 5 y 50 veces más rápido de leer dependiendo del dataset.

Rich para la terminal

Rich es una librería Python para crear interfaces de terminal elegantes y con información densa. Permite mostrar tablas, paneles con colores semánticos y actualizaciones en tiempo real sin las complejidades de una interfaz gráfica. Es lo que hace que ver los resultados trial a trial en terminal sea una experiencia limpia y profesional.

Excel para los resultados iniciales

El Excel tiene una ventaja enorme sobre otras alternativas para los resultados: cualquiera sabe abrirlo, filtrarlo, ordenarlo y añadirle sus propios gráficos. No requiere ninguna herramienta especial. En el futuro se migrará a SQLite para tener consultas más potentes, pero mientras el usuario esté aprendiendo cómo funciona el sistema, Excel es la opción más práctica.

HTML con Lightweight Charts para los gráficos

Los gráficos de resultados son archivos HTML standalone que usan Lightweight Charts, una librería de gráficos financieros de TradingView. No necesitan servidor ni conexión a internet. Se pueden abrir, compartir por email o archivar en local. Son interactivos: zoom, scroll, tooltips. Y son ligeros: un archivo HTML de un trial típico pesa menos de 500KB.

10. Hoja de ruta de construcción

El sistema se construye en fases ordenadas por dependencias: nunca se construye algo que depende de algo que aún no existe. Al final de cada fase hay algo funcional y verificable.

Fase 1 — La base de datos y la estructura

Objetivo al terminar esta fase: poder cargar un archivo Parquet, resamplearlo a cualquier timeframe y ver el resultado en pantalla. Nada más, pero bien hecho.

- Crear el árbol de carpetas completo del proyecto, vacío pero con su estructura definitiva.
- Escribir config.py con todos los parámetros y sus valores por defecto documentados.
- Implementar la carga de archivos Parquet, CSV y Feather con su validación correspondiente.
- Implementar el resampleo de OHLCV con todas las reglas de agregación correctas.
- Verificar manualmente que el resampleo es correcto comparando velas individuales.

> **CRITERIO DE AVANCE** No pasar a la Fase 2 hasta que el resampleo sea verificadamente correcto. Un resampleo incorrecto contamina todos los resultados posteriores y puede no detectarse hasta mucho más tarde.

Fase 2 — Las estrategias

Objetivo: una estrategia genera señales correctas y verificables sobre datos reales.

- Implementar la clase base de estrategias con todos sus helpers de indicadores.
- Implementar el auto-registro de estrategias por escaneo de carpeta.
- Escribir una estrategia de ejemplo simple (cruce de RSI) bien comentada, que sirva de plantilla.
- Verificar manualmente que las señales que genera la estrategia ocurren en las velas correctas y no en las siguientes ni en las anteriores.

Fase 3 — El motor Rust

Objetivo: simulate_trades() funciona correctamente con stop loss y take profit fijo.

- Crear el proyecto Rust con su estructura de archivos.
- Implementar el kernel de simulación con el loop vela a vela.
- Implementar la lógica de stop loss y take profit.
- Implementar la gestión de capital: colateral, apalancamiento, comisiones, saldo mínimo.

- Compilar y verificar que se puede importar desde Python.
- Verificar que las entradas ocurren siempre en el open de la vela siguiente a la señal.
- Verificar la reproducibilidad: misma entrada produce siempre la misma salida.

Fase 4 — Integración completa sin Optuna

Objetivo: un trial de punta a punta funciona: datos, estrategia, motor, métricas, resultado.

- Conectar la estrategia con el motor: generar señales → convertir a arrays → llamar al motor Rust.
- Implementar el cálculo de todas las métricas de rendimiento.
- Implementar la función de puntuación (score = ROI).
- Ejecutar un trial manual con parámetros fijos y verificar que los resultados tienen sentido.

Fase 5 — Optuna y los resultados

Objetivo: python run.py ejecuta 128 trials y genera Excel y HTML automáticamente.

- Implementar los tres modos de Optuna: QMC, TPE e HYBRID.
- Implementar el runner que coordina el bucle de trials con paralelismo configurable.
- Implementar el display de Rich en terminal: panel trial a trial con métricas en tiempo real.
- Implementar la generación del Excel con las tres pestañas.
- Implementar la generación de los HTML interactivos.
- Implementar run.py como punto de entrada único.
- Test completo de punta a punta.

Fase 6 — Multi-activo y multi-estrategia

Objetivo: configurar varios activos, timeframes y estrategias y que todo funcione correctamente con limpieza de RAM entre combinaciones.

- Actualizar run.py para gestionar múltiples combinaciones.
- Implementar el ciclo secuencial entre activos/timeframes con limpieza explícita de memoria.
- Verificar que cada combinación genera su carpeta de resultados independiente y correctamente nombrada.
- Verificar el uso de memoria con varias combinaciones en secuencia.

Fase 7 — Pulido del backtesting

Objetivo: el módulo de backtesting está completo y listo para uso diario intensivo.

- Implementar las salidas personalizadas de estrategia en el motor Rust.
- Gestión robusta de errores: si un trial falla, el run continúa sin detenerse.
- Rotación automática de archivos: cuando se supera el máximo configurado, se borran los más antiguos.
- Escribir la guía práctica de cómo crear una estrategia nueva (una página, didáctica).

Fases 8 y 9 — El resto de módulos

Una vez el backtesting funciona sólidamente en producción, se construyen los demás módulos en este orden:

Fase 8 — Portfolio Tracker	Base de datos SQLite con posiciones y capital. Terminal o script simple. Sin interfaz web todavía.
Fase 8 — Asset Scanner	Función de scoring inicial simple, lista de activos, actualización periódica, alertas básicas.
Fase 8 — Análisis	Gráfico interactivo con carga de estrategias validadas en backtesting.
Fase 8 — Noticias	API de noticias, calendario manual, sistema de notas personales.
Fase 9 — Servidor	Dashboard web para los módulos II-V. El backtesting ya corre en servidor desde el principio.
Fase 9 — Descarga de datos	Sección para descargar datos de Binance, Yahoo Finance u otras fuentes automáticamente.
Fase 9 — Score avanzado	Reemplazar el ROI simple por una función de puntuación más sofisticada.
Fase 9 — Perturbaciones	Anti-overfitting: introducir ruido aleatorio controlado en los datos durante el entrenamiento para que la estrategia sea más robusta.

PLATAFORMA DE INVERSIÓN — GUÍA DE DISEÑO

Este es un documento vivo.

Cada decisión importante de diseño que no esté aquí, debería estar.

El esqueleto primero. La velocidad siempre. La modularidad nunca se negocia.